



FIT2099 A3 DESIGN RATIONALE

CL_Lab08TeamC

Kian Lok Chin 35090294

Lim Chi Jian 35040424

Kar May Ong 34477241

Pemudi Hiruni

Halgahawatta 34246517

Liyanaarachch

Youssef Ahmed Adel 34073817

Attia Hassanein

Design Rationale

REQ1 – Limveld

A. Teleportation Mechanism Implementation

Design 1: Embedding Teleport Logic Directly in Actor Class

Pros	Cons
Simpler class hierarchy with no extra action subclasses	Actor handles both core and teleport logic
Direct access to Actor internals for teleport logic, enabling optimizations	Teleport logic tied tightly to Actor, limiting reuse in other contexts
	Any change to teleport requires Actor modification, increasing risk of bugs
	Code becomes cluttered as teleport logic grows

Design 2: Independent TeleportAction Class

Pros	Cons
Clear separation of concerns: teleport behavior isolated in TeleportAction class	Requires understanding of Action abstraction, adding conceptual overhead
Looser coupling allows reuse across different Actor types or map elements	Actors or map elements must explicitly register TeleportAction for functionality
Easier to extend or override teleport behavior via subclassing or composition	

B. Behaviour Selection Strategy

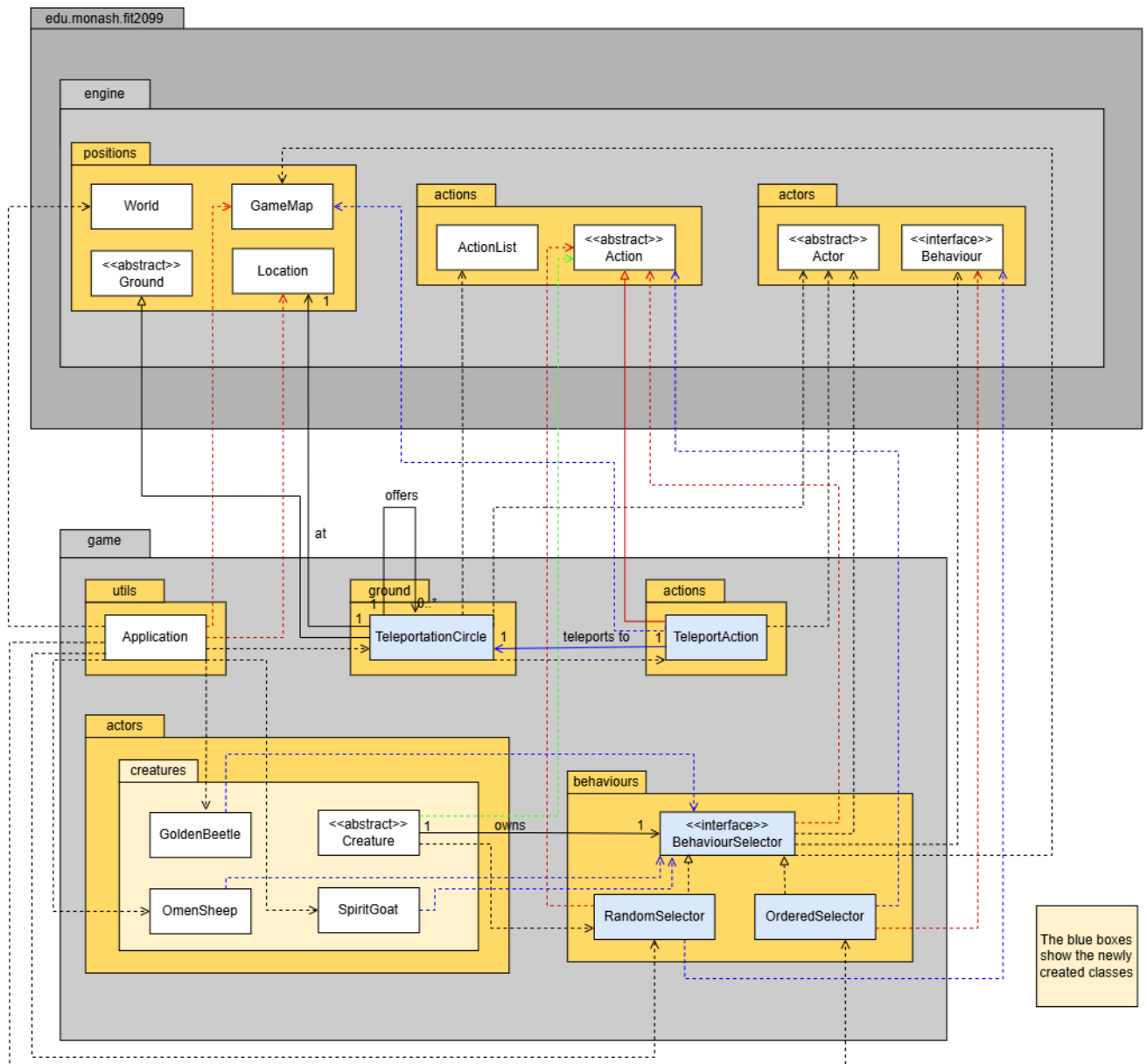
Design 1: Hardcoding Selection Logic in Actor

Pros	Cons
All decision logic centralized, making it quick to find and modify	Changes to logic require editing Actor code
No extra interfaces or design patterns to learn	High coupling reduces code reuse across different actor types

Design 2: BehaviourSelector Interface

Pros	Cons
Encapsulates decision logic, improving modularity and separation of concerns	More classes and interfaces increase codebase size
Supports easy extension by adding new selectors without modifying Actor	Possible over-abstraction for very simple actors
Enables reuse of selectors across different Actors or NPC types	

UML REQ1



The selected approaches are as follows: for the **Teleportation Mechanism**, Design 2 was chosen, which uses an independent TeleportAction. The **Behaviour Selection Strategy** adopts Design 2, utilizing a BehaviourSelector interface.

Using protected access modifier for both behaviours and selector in the Creature class allows subclasses like OmenSheep and SpiritGoat to directly configure and modify their behavior logic. This supports flexible customization without exposing internals to unrelated classes, promoting good encapsulation while enabling extensible and readable subclass design without unnecessary boilerplate methods.

Apart from that, `public static final int EGG_TIMER_THRESHOLD` is declared `public` so that external classes—like the `Application` class—can access it. This allows the game to reference

the Omen Sheep's default egg timer value (e.g., for display, configuration, or debugging) without instantiating the class. It promotes reusability and consistency.

This design adheres closely to the **Single Responsibility Principle SRP**. TeleportAction is solely responsible for the teleport logic, allowing actors to remain focused on their own state and executing actions. Meanwhile, the TeleportationCircle manages only a list of destination teleportation circles and generates corresponding TeleportAction instances. The BehaviourSelector interface and its implementations are dedicated exclusively to deciding “which behaviour to pick,” without mixing behaviour execution logic.

The **Open/Closed Principle OCP** is maintained by enabling new teleport variants, such as a “TeleportWithParticleEffect,” to be added through subclassing or wrapping TeleportAction. This means existing classes remain untouched. Similarly, new behaviour-selection strategies like WeightedSelector or CompositeSelector can be introduced simply by implementing the BehaviourSelector interface, without modifying existing actors or selectors.

The design follows the **Liskov Substitution Principle LSP** since any class implementing BehaviourSelector can seamlessly replace another in an actor’s constructor. Likewise, any subclass of TeleportAction can substitute the base class without breaking existing teleportation calls, as they adhere to the same Action contract.

In terms of the **Interface Segregation Principle ISP**, BehaviourSelector contains only the method selectAction(...), so actors aren’t forced to depend on unused methods. If more complex behaviour selection is required in the future (e.g., bulk evaluation), separate interfaces can be introduced without bloating the current one.

Regarding the **Dependency Inversion Principle DIP**, actors depend on the abstraction BehaviourSelector rather than any concrete implementations like RandomSelector or OrderedSelector. Similarly, map or ground classes depend on the Action interface for teleportation, rather than any specific teleport action class, which enhances flexibility and testability.

The design also embraces **DRY (Don’t Repeat Yourself)** principles by centralizing all teleport logic inside the TeleportAction class. If teleport behaviour changes (for example, by adding a mana cost), only that class needs modification. Behaviour-selection logic is not duplicated across actors since all actors reuse the same selector implementations.

Connascence Minimization is achieved by ensuring that TeleportAction and TeleportationCircle depend only on Location and Action, without relying on the internals of Actor or GameMap. Similarly, BehaviourSelector implementations depend only on the behaviour contract, not on specific behaviour details. Actors only need to know that teleport

or selection strategies conform to their abstract interfaces, without knowing their implementation details.

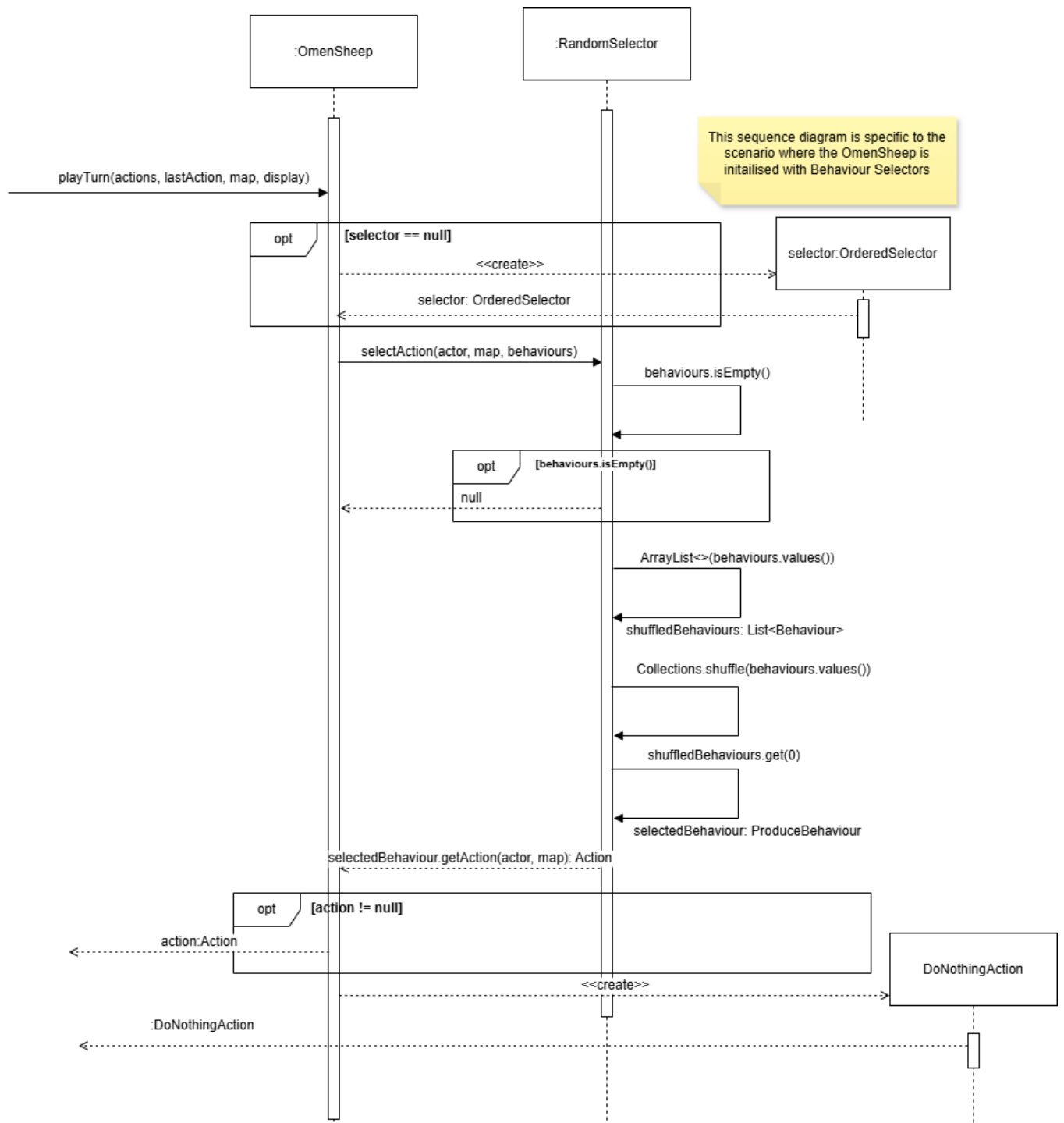
In this design, the primary types of connascence present are **Connascence of Name (CoN)** and **Connascence of Type (CoT)**, as components like `TeleportAction`, `TeleportationCircle`, and `BehaviourSelector` must consistently reference and use the same class and interface names (e.g., `Location`, `Action`, `BehaviourSelector`) and rely on their specific type signatures. Additionally, there is **Connascence of Identity (CoI)** between `Actor` and implementations of `BehaviourSelector`, since all parties must conform to the same method contract (`selectAction(...)`). More fragile forms like **Connascence of Position (CoP)** and **Connascence of Meaning (CoM)** are avoided due to the clean separation of concerns and use of well-defined abstractions, which helps minimize tight coupling and enhances maintainability.

Future Maintainability & Extensibility are well supported. New teleport variants, such as a portal that requires a key or has a cooldown, can be created by subclassing or decorating `TeleportAction`. Additional behaviour selectors can be implemented, such as a `WeightedSelector` that chooses based on behaviour weights, or a `ContextualSelector` that factors in actor health or distance. Actors can easily switch strategies by injecting different selectors.

For **Concurrency**, if teleport targets might be modified at runtime by multiple threads (due to dynamic world changes), the `List<TeleportationCircle>` can be replaced with a thread-safe collection or synchronized during modification.

By combining an independent `TeleportAction`, a simple yet flexible target list, and a strategy-based behaviour-selection interface (with both random and ordered implementations), this design ensures clear separation of concerns, strong adherence to SOLID principles, minimal coupling, and maximum extensibility for future game features.

Sequence Diagram REQ1



REQ2 – BedOfChaos

A. Growable Component Architecture

Design 1: Single Class with Internal Conditional Logic

Pros	Cons
Combining logic into one method reduces class count and initial setup.	Mixes parent and component creation logic, complicating maintenance.
Centralized growth logic ensures consistency and easier debugging.	Adding new component types requires modifying existing methods, hurting extensibility.
	Hard to reuse component behaviors independently due to tight coupling.

Design 2: Composite Pattern via a Growable Interface with Separate Classes

Pros	Cons
New component types can be added by implementing the Growable interface without altering existing classes.	Extra interfaces and classes can feel like boilerplate if behaviors are simple, increasing code size.
Clear separation: Leaf heals, Branch handles branching, BedOfChaos manages collections, improving clarity.	

B. BedOfChaos Actor and Behaviour Composition

Design 1: Hardcoded Priority Loop Over Behaviours

Pros	Cons
No Extra Abstractions Needed: Designers can see the behaviour order clearly, without needing a separate selector or strategy class, reducing initial complexity.	Adding a new behaviour or changing priority requires editing existing constructor logic.
	Other actors requiring similar behaviour logic must duplicate or refactor the loop.

Design 2: Strategy Pattern via a BehaviourSelector Interface

Pros	Cons
New selection strategies can be added without changing the actor class.	Adds abstraction overhead that may be unnecessary if only one strategy is used, increasing setup complexity.
The actor depends only on the BehaviourSelector abstraction, not on any specific selection algorithm, enhancing modularity.	Requires injecting a selector during actor creation, increasing setup complexity and risk of misconfiguration.
Selector logic can be reused across multiple actors, reducing duplication and supporting DRY principles.	

C. BedOfChaos Damage Point Calculation

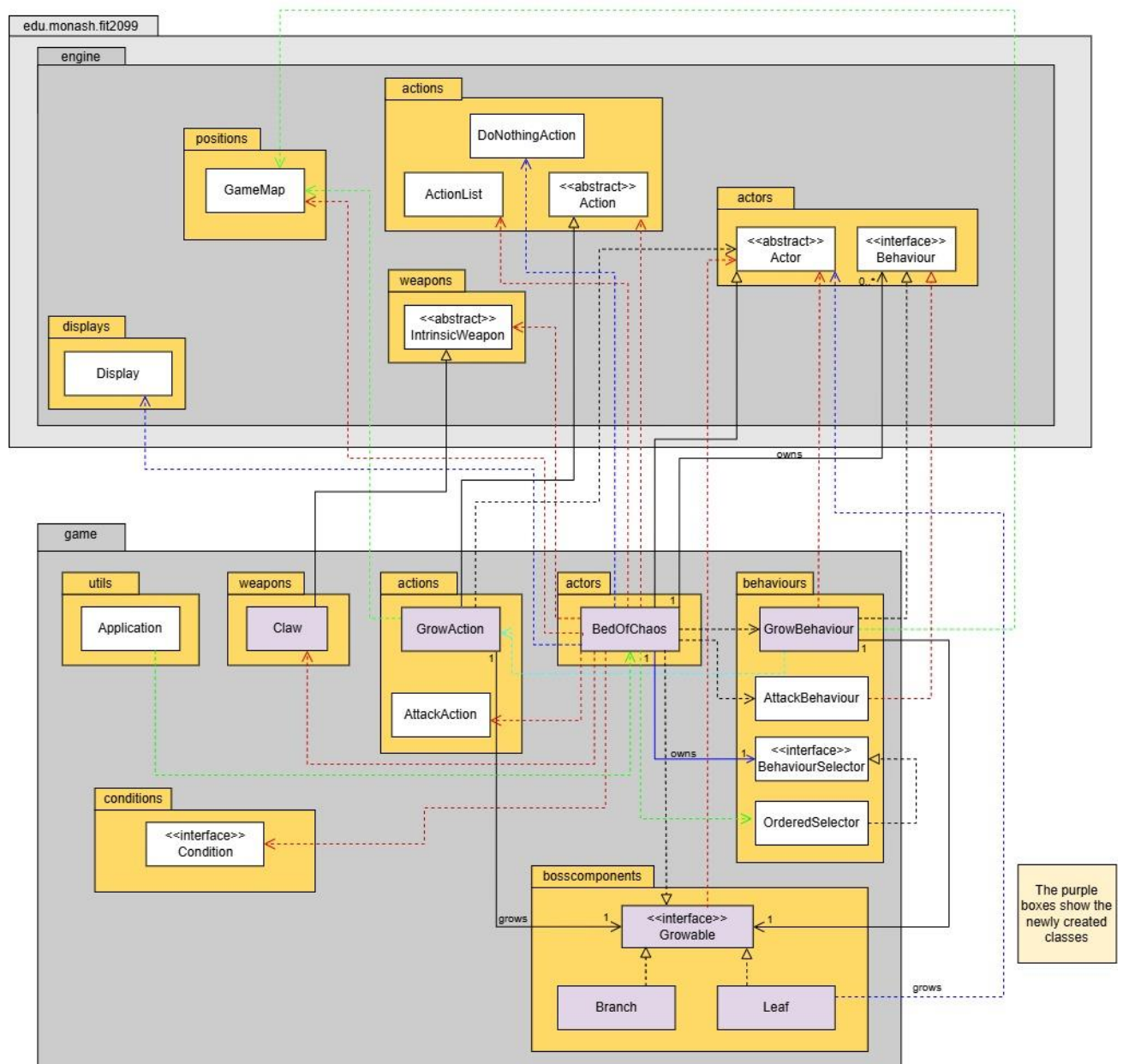
Design 1: Create 2 fields to keep track of the number of branches and leaves grown

Pros	Cons
Simple to understand and implement if there are only 2 components	Hard to reuse if new components are introduced due to tight coupling

Design 2: Use a list to store the components grown from BedOfChaos

Pros	Cons
Highly extensible as new components can be added without modifying existing code	Performance overhead as damage need to be recalculated every time
Allow damage point of its recursive components to be calculated recursively	

UML REQ2



The selected design for the **Growable Component Architecture** was Design 2, utilizing a **Growable** interface with specific implementations such as **Branch**, **Leaf**, and **BedOfChaos**. The **Behaviour Composition in BedOfChaos** follows Design 2, which uses a strategy pattern via a **BehaviourSelector** Interface. The **Damage Point Calculation for BedOfChaos** follows Design 2, which uses a list to store its components grown.

The reason we inject **BehaviourSelection** pattern inside **BedOfChaos** is to enhance the extensibility of our code, allowing behaviour to be selected with different strategy.

Each class focuses on one responsibility which strictly follows the **Single Responsibility Principle SRP**. **Claw** defines only weapon attributes like damage and verbs. Every **Growable** implementation handles its specific growth logic, while **BedOfChaos** aggregates them without

accessing internals. Similarly, AttackBehaviour is solely concerned with deciding when to attack, deferring condition checks to separate classes. BedOfChaos.playTurn() orchestrates the behaviour order but doesn't manage their individual logic.

Open/Closed Principle (OCP): The design supports extension without modifying existing code. New weapon variants like FieryClaw can subclass Claw. Additional growable components (e.g., Thorn, Flower) can implement Growable without affecting existing code. Behaviours and conditions can be expanded independently LowHealthCondition can added to the system without altering existing structures when the boss only attacks when Player is low on health.

Liskov Substitution Principle (LSP): Subclasses like FireClaw or new Growable components can seamlessly replace their base types without breaking functionality. AttackBehaviour works with any Condition that meets the expected interface, preserving substitutability. BedOfChaos's component list that stores Growable allow any Growable object to be used seamlessly in place.

Interface Segregation Principle (ISP): Interfaces are focused and minimal. Condition includes only isSatisfied(), and Growable exposes only necessary methods. This prevents classes from depending on methods they do not use.

Dependency Inversion Principle (DIP): High-level modules depend on abstractions, not concrete implementations. AttackBehaviour works with any Condition, and BedOfChaos manages a list of Growable interfaces. The sorted map of behaviours also decouples BedOfChaos from specific behaviour logic.

Don't Repeat Yourself (DRY): Logic is centralized and reused. Growth logic is encapsulated within each respective class, and Claw manages its attack attributes without needing duplication elsewhere.

Connascence Minimization:

Classes communicate through interfaces rather than depending on internal structure. BedOfChaos does not know how each behaviour works, only that it provides an Action. Growable components function independently and are aggregated uniformly.

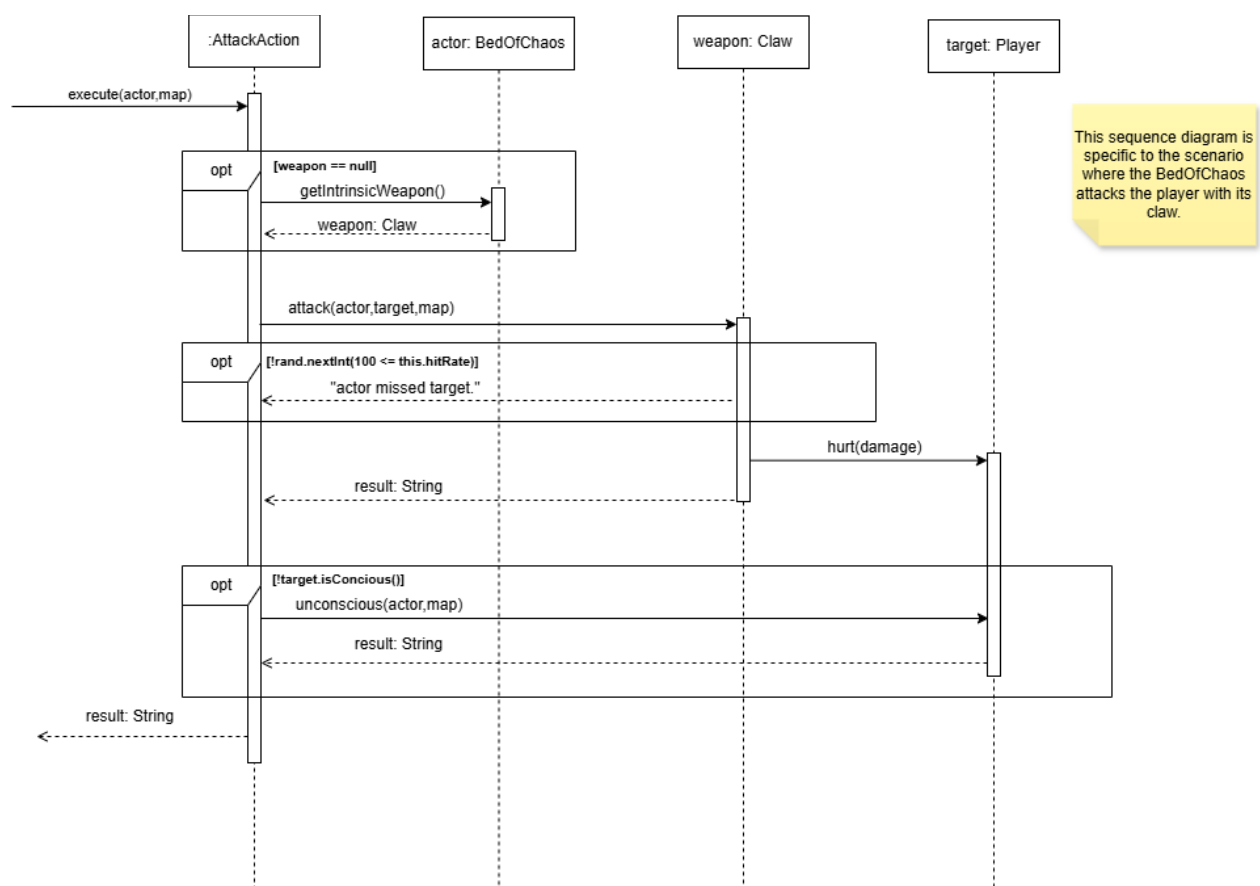
In this design, the primary types of **connascence** are **Connascence of Name (CoN)** and **Connascence of Type (CoT)**, as components such as Growable, Condition, and Claw rely on shared class and interface names and types to function correctly such as e.g., BedOfChaos interacts with Growable and Behaviour through well-defined type contracts. The design explicitly **minimizes stronger forms** like **Connascence of Meaning (CoM)** or **Connascence of Position (CoP)** by using abstraction boundaries and clearly named, minimal interfaces. For example, BedOfChaos does not depend on the internal workings of each Growable or

behaviour class as it only depends on what their interfaces promise, promoting loose coupling and easier maintenance.

The system is highly extensible. New intrinsic weapons like PoisonClaw can be added by subclassing. Additional Growable types can be introduced with minimal impact. Composite conditions like “low stamina AND low health” can be created without touching existing behaviour logic. Moreover, we can add more boss components that are growable by just implementing the Growable interface which has grow() that allows implementation of effects of the growing process such as hurting BedOfChaos and getDamagePoint() which allows negative and positive damage accumulation. For instance, in the future we might have PoisonedLeaf which will reduce the damage of the boss and hurt the boss when growing.

By adopting a clean, modular structure with dedicated classes like Claw, Growable, and Condition, this design achieves strong separation of concerns. It adheres to SOLID principles and lays a robust foundation for future extensions, improvements, and gameplay evolution.

Sequence Diagram REQ2



REQ3 – Weather Madness

A. Weather Effect Implementation

Design 1: Implement all weather effects in one class

Pros	Cons
Simple to implement; all logic is centralized in a single class	Strong coupling between weather effects, making extension difficult
No additional abstraction, easier for beginners to understand	Each new weather effect requires modifying the class, making the system harder to extend without changing existing code.

Design 2: Use a WeatherEffect abstract class with polymorphism

Pros	Cons
Each weather effect is its own class, making the system easier to extend and maintain	If multiple weather effects have similar logic, they need to be updated manually, which is time-consuming.
New effects can be added without modifying the existing code, making the system easier to extend.	

B. Environmental System Design Comparison

Design 1: All-in-One/Direct Control in Player

Pros	Cons
Simple to implement, all logic in one place	Strong coupling between environment, weather, and player logic
Easy to trace for small projects	Adding new environment effects requires modifying core classes, making the system harder to maintain and extend.

Design 2: Decoupled EnvironmentalStatusSystem

Pros	Cons
The environmental logic is separated from weather and player, improving modularity and clarity.	Initial setup may require more effort
Easy to extend as new environment effects can be added without modifying core classes	Requires understanding of event/listener or observer pattern
Promotes loose coupling and modularity	
Supports future expansion (e.g., multiple listeners, more complex environment interactions)	

C. Item Use Logic Design Comparison

Design 1: Separate UseTorchAction & UseUmbrellaAction Classes

Pros	Cons
The item class focuses solely on describing the item, while the action class manages its behavior.	The codebase will have more files, which may seem overwhelming at first.
Promote code reuse and modularity	Requires understanding of action patterns and class collaboration: Developers need to be familiar with how actions and items interact.
Easy to add new actions or modify existing ones without changing item classes	

Design 2: Direct Logic in Torch/Umbrella Classes

Pros	Cons
Simple and straightforward: All logic is written directly inside the item class, making it easy to see how the item behaves at a glance.	The torch/umbrella class handles too many responsibilities, making it harder to maintain and understand.

The system design adheres to **the Single Responsibility Principle SRP** by assigning each component to a focused role. In the Weather Effect implementation, an abstract WeatherEffect class and subclasses like AcidRainEffect or SnowEffect encapsulate weather-specific behavior, keeping that logic separate from the broader weather system or game framework. The Environmental Status System is decoupled from both player and weather logic, managing environmental changes (e.g., temperature shifts) independently. In Item Use logic, Torch and Umbrella classes maintain only item state (e.g., whether a torch is lit), while UseTorchAction and UseUmbrellaAction encapsulate usage behavior, preventing item classes from handling both state and action.

Extensibility without altering the existing code is ensured by the **Open-Closed Principle (OCP)**. New weather effects—such as a FogEffect that reduces visibility can be introduced simply by creating a new WeatherEffect subclass implementing applyEffect , with no changes to existing classes. The Environmental Status System employs an observer pattern, so additional environmental effects (e.g., wind, radiation) are added via new listener implementations rather than modifying the core system. In Item Use logic, adding a new action—say UseLanternAction for a lantern—requires creating its own class without touching existing Torch or Umbrella classes, since action logic is already encapsulated in separate action classes.

Liskov Substitution Principle (LSP) compliance is maintained by ensuring all subclasses conform to their abstract base contracts. Any WeatherEffect subclass (e.g., AcidRainEffect, SnowEffect) can substitute for WeatherEffect without altering expected behavior when applied to actors or environments. Environmental Status System listeners adhere to a common listener interface, so any listener can be interchanged without side effects. In Item Use logic, UseTorchAction and UseUmbrellaAction implement a shared Action interface, ensuring interchangeable behavior within the game’s action framework.

Interface Segregation Principle (ISP) is respected by avoiding bloated interfaces. The WeatherEffect interface defines only applyEffect, so subclasses implement only necessary weather behavior. The Environmental Status System uses specific listener interfaces (e.g., weather event listeners), preventing implementers from providing unused methods.

Dependency Inversion Principle (DIP) is evident in how higher-level modules depend on abstractions. Game logic invokes weather effects through the WeatherEffect abstraction rather than concrete implementations. The Environmental Status System notifies listeners via an abstract interface, remaining decoupled from specific listeners. Similarly, the game engine invokes item usage through the generic Action interface rather than directly referencing UseTorchAction or UseUmbrellaAction.

The design follows the **DRY (Don’t Repeat Yourself)** principle by centralizing common logic. Shared behaviors—like applying damage or buffs—reside in the WeatherEffect abstract class

or shared utilities and are reused by all weather effect subclasses. Environmental logic is consolidated within the Environmental Status System, avoiding duplication in player or weather classes.

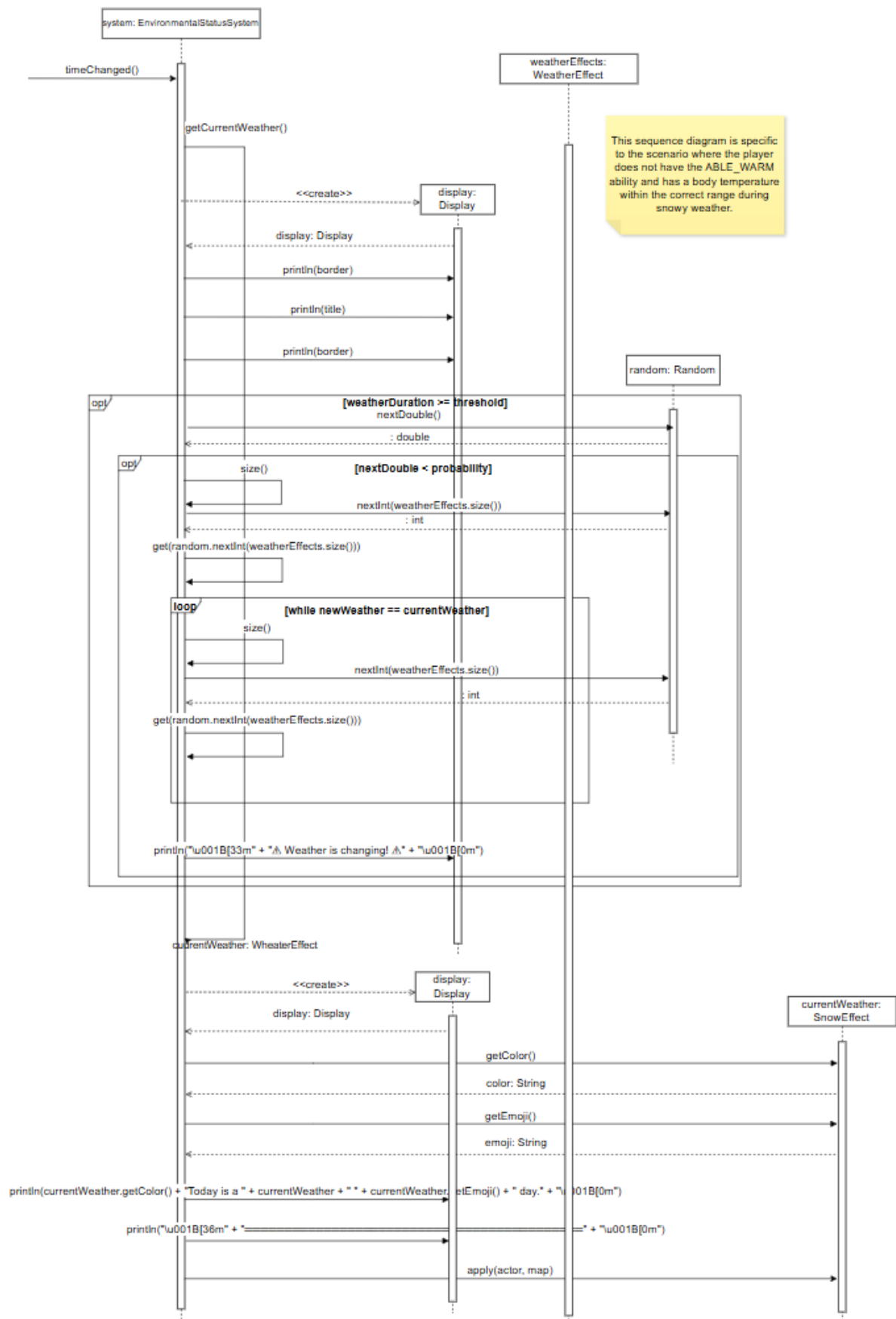
Connascence minimization reduces interdependence. WeatherEffect subclasses operate through a generic interface and do not require knowledge of actor internals. The Environmental Status System notifies listeners without understanding their implementations. In Item Use logic, Torch and Umbrella classes interact with action classes only via public methods (e.g., use, isLit), preventing reliance on each other's internal details.

In this design, the dominant forms of **connascence** are **Connascence of Name (CoN)** and **Connascence of Type (CoT)**, as components such as WeatherEffect, Action, and EnvironmentalStatusSystem must share consistent type names and signatures to interoperate—for instance, all weather effect classes must implement applyEffect() with the correct parameters, and item use actions must conform to the Action interface. Crucially, the design avoids tighter coupling by using polymorphism and abstract interfaces, thus preventing **Connascence of Meaning (CoM)** or **Position (CoP)**, which would have arisen if shared logic were encoded in positional or implicitly understood structures. This intentional separation ensures components depend only on necessary contracts, reducing fragility and enabling smooth evolution of the system.

Maintainability and extensibility are key strengths. Introducing a new weather effect—such as StormEffect imposing movement penalties—requires only new WeatherEffect subclass. Adding a radiation zone involves creating a listener and registering it with the Environmental Status System.

By adopting Design 2 for Weather Effect, Environmental System, and Item Use logic, the architecture remains flexible and scalable. Weather types can be added without altering core logic; the decoupled Environmental Status System supports modular growth; and separate action classes keep item behaviors reusable and testable. This makes the system robust in a dynamic game environment where items like Torch or Umbrella interact with weather conditions —acid rain affecting torch durability or requiring an umbrella—while remaining aligned with modern software engineering principles.

Sequence Diagram REQ3



REQ4 – Witch Mayhem of Potions

A. Potion Effect Implementation

Design 1: Abstract Potion Class with Subclasses and StatusEffect Objects

Pros	Cons
The potion itself (Potion) is separated from its effects (StatusEffect), ensuring distinct responsibilities and improving modularity.	Each New Effect Requires a StatusEffect Subclass: Creating a new effect (e.g., SpeedEffect) necessitates a new StatusEffect subclass, increasing the number of classes to manage.
Adding a new potion effect only requires a new Potion subclass and corresponding StatusEffect subclass, adhering to the Open-Closed Principle OCP.	
The same StatusEffect (e.g., PoisonEffect) can be triggered by different potions or items (if designed to allow this), promoting code reuse and DRY.	

Design 2: Hardcoded Effect Logic in Each Specific Potion Class

Pros	Cons
Direct for Simple Effects: For very simple effects (e.g., instant healing), hardcoding logic in the potion class is straightforward, reducing the need for additional classes.	Effect logic is bound to the potion class, limiting reuse in other contexts (e.g., non-potion items) and reducing modularity.
Minimal Initial Setup: Requires fewer classes and abstractions, allowing rapid prototyping for small systems with basic effects.	The potion class manages both identity and effect logic, mixing concerns and hindering maintainability.
	Similar effects (e.g. StrongHealingPotion) can lead to duplicated logic, violating DRY and increasing maintenance burden.

B. Potion Brewing System

Design 1: Modular Brewing via Pouch, BrewPotionAction and Potion classes

Pros	Cons
Pouch interconnects Brewing action and potions with Player class, while BrewPoitionAction executes brewing, ensuring modular and maintainable code.	Everytime a new Potion is added we need create more classes which lengthen the codebase.
Defining recipes as Status lists inside Potion classes enables easy modification and supports a dynamic, extensible crafting system.	Increase abstraction layer due to Pouch class.
Shared material validation and consumption logic in Pouch reduces duplication and adheres to DRY principles.	
New potions can be added without modifying existing code, aligning with the Open/Closed Principle.	

Design 2: Player Class Directly Handles All Brewing Logic with Hardcoded Item Names

Pros	Cons
Consolidating brewing logic in Player can be simple and quick for small systems with few potions.	Brewing logic is tightly bound to Player and specific items, limiting modularity and flexibility.
Allows rapid implementation during early development with basic brewing needs.	Adding new recipes requires modifying complex if-else structures, increasing error risk and maintenance overhead
	Material checks and consumption are repeated across branches, violating DRY principles.

C. Meat Ingredients Implementation

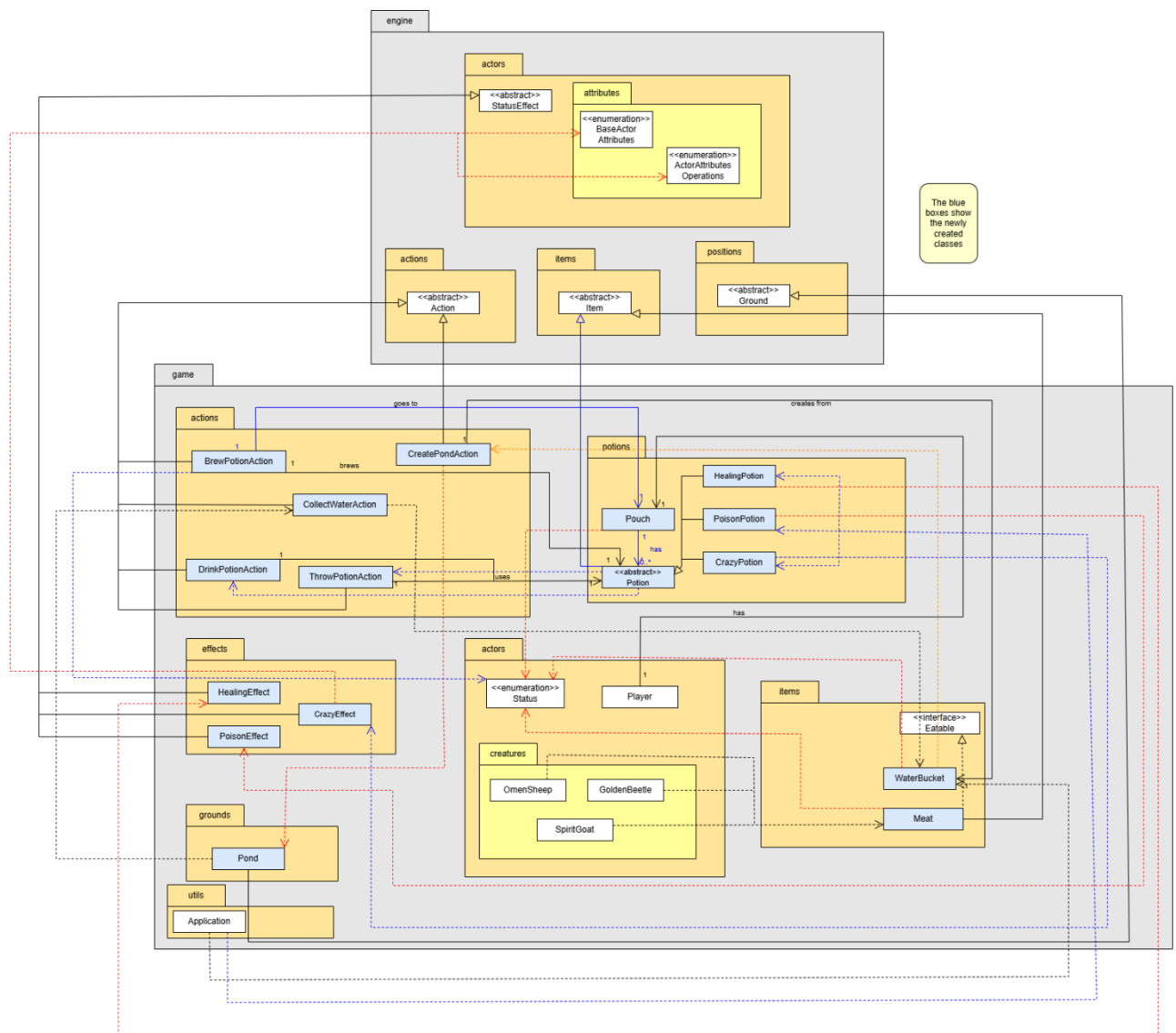
Design 1: Use Separate meat class to represent each creature meat along with their specified Status

Pros	Cons
Interaction logic (like EatAction) can be embedded directly in the Player or game loop, making quick prototyping easier.	Core behavior is dispersed across classes like Player, reducing cohesion and increasing maintenance difficulty.
New meat types can be added quickly without refactoring other classes.	As the system grows, managing individual meat subclasses becomes redundant and harder to maintain.

Design 2: Use a single Meat class with Status passed in as parameter

Pros	Cons
New meat can be added through new Status or Action classes, avoiding changes to existing logic (Open/Closed Principle).	Requires understanding of capabilities (Status) and how they're used for behavior control.
Only one Meat class is needed, avoiding unnecessary subclasses.	
Common logic for different meats (e.g., portable, edible) is centralized.	

UML REQ4



The diagram above illustrates the final design of **Requirement 4**, which applies **Design 1** for the **Potion Effect and Brewing System**, and **Design 2** for the **Meat Ingredients Implementation**.

Having separate Effect and StatusEffect classes is good object-oriented design because it cleanly separates **instantaneous effects** (like HealEffect) from **ongoing, time-based effects** (like HealingEffect). This distinction improves **clarity**: developers can easily tell whether an effect happens once or persists over time. It also enhances **flexibility and extensibility**, allowing you to add new types of effects without overcomplicating a single class.

The Effect interface handles one-off logic through its `apply()` method, making it suitable for consumable items or immediate game events. Meanwhile, `StatusEffect` defines behavior via a `tick()` method, designed to be called each turn—ideal for effects like poisoning or regeneration. Keeping these concerns separate aligns with the **Single Responsibility Principle**, making the codebase easier to maintain and debug. Additionally, it allows the engine to process each type efficiently and correctly within their respective game loops.

For the meat system, a **single Meat class** is used, with name and Status passed into the constructor. This approach provides flexibility while avoiding deep abstraction hierarchies. Variants of meat with different brewing statuses can be created easily without needing separate subclasses. This design is both **highly extensible** and supports **dependency injection**, as behaviour is determined by the passed-in status rather than hard-coded logic.

The design adheres strongly to the **Single Responsibility Principle (SRP)**. Each `StatusEffect` subclass encapsulates specific effect behaviour. Potion subclasses manage potion-specific data such as name, effects, and recipes. The Pouch class is responsible solely for tracking materials and recipes, while `BrewPotionAction` handles the brewing operation independently. Each class has a focused, well-defined responsibility.

We also comply with the **Open/Closed Principle (OCP)**. New potions or effects can be introduced by simply creating new subclasses without modifying existing code. Adding a new brewing recipe requires only its initialization in the Application class and registration with the Pouch—existing classes remain untouched.

The **Liskov Substitution Principle (LSP)** is respected throughout the design. Any subclass of `StatusEffect` (e.g., `PoisonEffect`, `SpeedEffect`) can be used wherever a `StatusEffect` is expected. Similarly, item-interaction Action subclasses can be passed via `allowableActions` and function correctly without altering the consumer's logic.

The **Interface Segregation Principle (ISP)** is applied by keeping interfaces lean and focused. The `StatusEffect` interface includes only essential methods like `tick()` and `toString()`. The Pouch class exposes only the methods relevant to material checking and consumption, while `BrewPotionAction` implements only what's necessary for the brewing process—no class is burdened with irrelevant responsibilities.

Dependency Inversion Principle (DIP) is reinforced by depending on abstractions. For instance, the Player interacts with items through abstract Action classes, not their concrete implementations. The brewing system interacts with the abstract `BrewPotionAction` and Pouch interfaces rather than specific recipes. Likewise, the potion system relies on the `StatusEffect` abstraction instead of being tightly coupled to concrete subclasses.

The design follows the **DRY (Don't Repeat Yourself)** principle by centralizing shared logic. All potion effects are implemented once in `StatusEffect` subclasses and reused across multiple potions. Material checking is implemented in `Pouch` and reused by `BrewPotionAction`.

From a connascence perspective, the design exhibits mostly **Connascence of Name (CoN)** and **Connascence of Type (CoT)**. Classes like `Potion` and `StatusEffect` agree on method names and expected types (e.g., `tick()`), which are manageable through strong interface contracts. More fragile forms like **Connascence of Position (CoP)** or **Meaning (CoM)** are avoided by clearly naming parameters and encapsulating logic within isolated components.

Overall, the system is built for **long-term maintainability** and **modular growth**. Adding new features—such as a new potion effect (`RegenerationEffect`) or a new creature that drops meat—requires no modification to existing logic. Even enabling future NPCs such as wizards to brew potions is seamless: simply inject a `Pouch` during instantiation via the `Application` class.

This architecture demonstrates strong alignment with **SOLID principles**, resulting in a clear separation of concerns, low coupling, and high cohesion. It lays a robust, extensible foundation that allows the system to evolve organically without unnecessary complexity or refactoring.

Sequence Diagram REQ4

